
**Faculty of Engineering and Technology**

**Electrical and Computer Engineering Department**

# Operating System

## ENCS3390

## First Semester 2024/2025

## Project #1

**Instructor:** Dr. Abdel-Salam Sayyad


| Name | ID | Section |
|---|---|---|
| Sari Abdal-ghani | 1220982 | 3 |


**Submission Date: 8/12/2024**

# Table of Contents

# Table of Figures

# 1. Theory

## 1.1 Multiprocessing:

"Multiprocessing refers to a system that has more than two central processing units (CPUs). Every additional CPU added to a system increases its speed, power and memory. This allows users to run multiple processes simultaneously. Each CPU may also function independently, and some CPUs may remain idle if they don't have anything to process. This can improve the reliability of a system because unused CPUs can act as a backup if technical issues arise."[1]

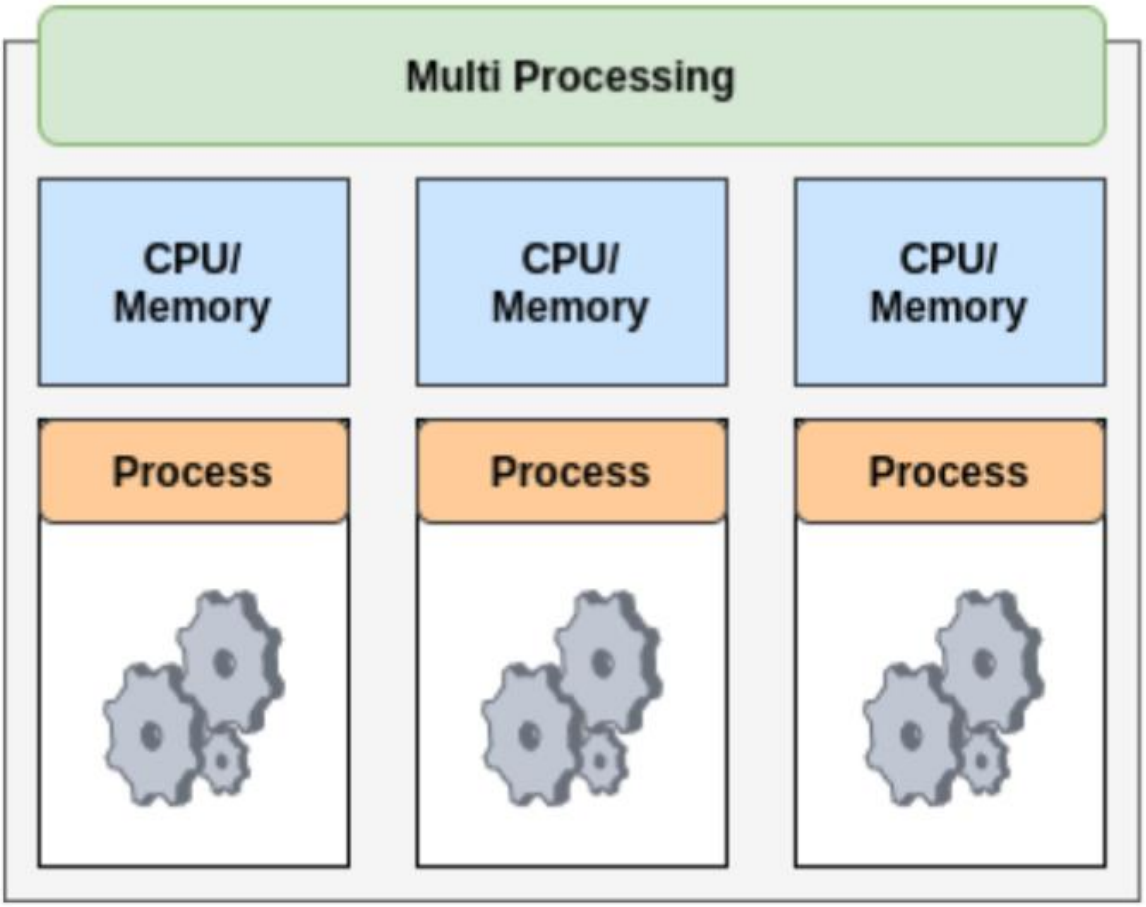


Figure 1: Struct of Multiprocessing

## 1.2 Multithreading:

“Multithreading is a programming technique that assigns multiple code segments to a single process. These code segments, also referred to as threads, run concurrently and parallel to each other. These threads share the same memory space within a parent process. This saves system memory, increases computing speed and improves application performance. For example, if you're working on your computer, you might have multiple browser tabs open while searching the internet. You might also be listening to music through a desktop application at the same time. The internet browser and music application represent two different processes, even though they are operating simultaneously. However, the multiple tabs you have open while browsing the internet represent threads of your internet browser, which is the parent process.”[1]

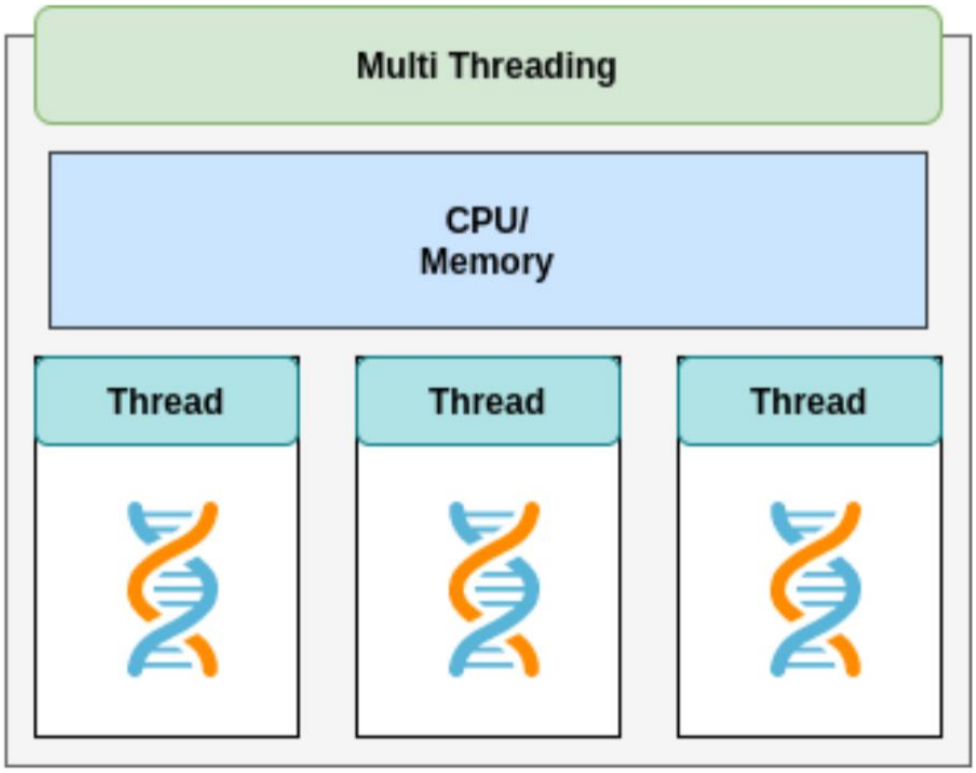


Figure 2: Struct of Multithreading

# 2. Procedure:

## 2.1 My laptop specifications:

I was use virtual box to do the project, first I will The show details for my laptop:

- ⇨ Cores: 4 real core ( 8 logical core)
- ⇨ Speed: 2.80 GHz
- ⇨ Ram: 8 GB
- ⇨ Rom: 512GB (SSD)
- ⇨ OS: windows 11 pro

For virtual box (virtual machine) I was give it the following:

- ⇨ Cores: 8 cores
- ⇨ Ram: 8 GB
- ⇨ Rom: 20GB
- ⇨ OS: Ubuntu
- ⇨ Programing language: C program
- ⇨ IDE tool: Gedit in Terminal

## 2.2 achieved the multiprocessing and multithreading

- The Naive approach: for my code first I read the file into array 2D, then built an array of struct that have the unique words and its Frequency, after that the code will check the words in the 2D array if it exists in the unique words array then will increment the frequency, if not then will insert new word to unique words array. Then the program will search for the most 10-word frequency and print them.

After built the Naïve code then I can build the multiprocessing and multithreading dependent on the naïve code:

➔ For Multiprocessing code: in my program I was use maltiprocess in calculate the frequency for unique words from array of 17010000 index, so I divide the big array into the process and each process was doing its part, in addition I used in my program pipes to send the data from children to parent, and merge the data from children in the unique array.

- To make new child use fork() function from unisted.h library.
- To make pipe use pipe() function from unisted.h library.
- To wait use wait() function from sys/wait.h library.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define SizeOfWords 17010000
#define StringSize 90
#define SizeOfUnique 300000
#define NumProcess 2

struct words {
```

Figure 3: The library that I use in multiprocessing

```c
for (int p = 0; p < NumProcess; p++) {
    int pid = fork();

    if (pid == 0) { // Child process
        close(fd[p][0]); // Close reading

        struct words *childFreqWords = (struct words *)malloc(SizeOfUnique * sizeof(struct words));
        int childNumOfUnique = 0;

        // Calculate the range of words for this process
        int start = p * (NumOfWords / NumProcess);
        int end;
         if(p == NumProcess - 1)
            end=NumOfWords ;
         else
            end=start + (NumOfWords / NumProcess);

        // Child process work in its part from the word array
        for (int i = start; i < end; i++) {
            int k;
            for (k = 0; k < childNumOfUnique; k++) {
                if (strcmp(array[i], childFreqWords[k].string) == 0) {
                    childFreqWords[k].freq++;
                    break;
                }
            }
            if (k == childNumOfUnique) {
                strcpy(childFreqWords[childNumOfUnique].string, array[i]);
                childFreqWords[childNumOfUnique].freq = 1;
                childNumOfUnique++;
            }
        }

        // Write results to pipe
        write(fd[p][1], &childNumOfUnique, sizeof(int));
        write(fd[p][1], childFreqWords, childNumOfUnique * sizeof(struct words));
```

Figure 4: create process and divide the array

➔ For Multithreading for each thread I was find the start index and end index from the main array then in calculate the frequency for unique words. In addition, I use mutex to lock before increment frequency or adding unique word and unlock after that.

- To create new thread use pthread_create() function from pthread.h library.
- To join the thread use pthread_join() function from pthread.h library.
- To lock data use pthread_mutex_lock () function from pthread.h library.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define SizeOfWords 17010000
#define StringSize 90
#define SizeOfUnique 300000
#define NumThreads 8

```

Figure 5: The library that I use in multithreading

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *calculateFreq(void *arg) {
    struct ThreadInfo *data = (struct ThreadInfo *)arg;
    char **array = data->array;
    int start = data->start;
    int end = data->end;
    struct words *freqWords = data->freqWords;
    int *numOfUniqueWords = data->numOfUniqueWords;
    pthread_mutex_t *mutex = data->mutex;

    for (int i = start; i < end; i++) {
        int k;
        for (k = 0; k < *numOfUniqueWords; k++) {
            if (strcmp(array[i], freqWords[k].string) == 0) {
                pthread_mutex_lock(mutex); // Lock before modifying
                freqWords[k].freq++;
                pthread_mutex_unlock(mutex); // Unlock after modification
                break;
            }
        }

        // If the word is not found in freqWords
        if (k == *numOfUniqueWords) {
            pthread_mutex_lock(mutex); // Lock before modifying
            strcpy(freqWords[k].string, array[i]);
            freqWords[k].freq = 1;
            (*numOfUniqueWords)++;
            pthread_mutex_unlock(mutex); // Unlock after modification
        }
    }

    return NULL;
}
```

Figure 6: calculate the frequency and lock data in multithreading

## 2.3 Amdahl's law:

- To calculate the percentage is the serial part of my code I need to use clock from time.h library, in my code all the code is serial expect calculate frequency for unique words so I will calculate the time before it and added it to the time after calculate frequency:

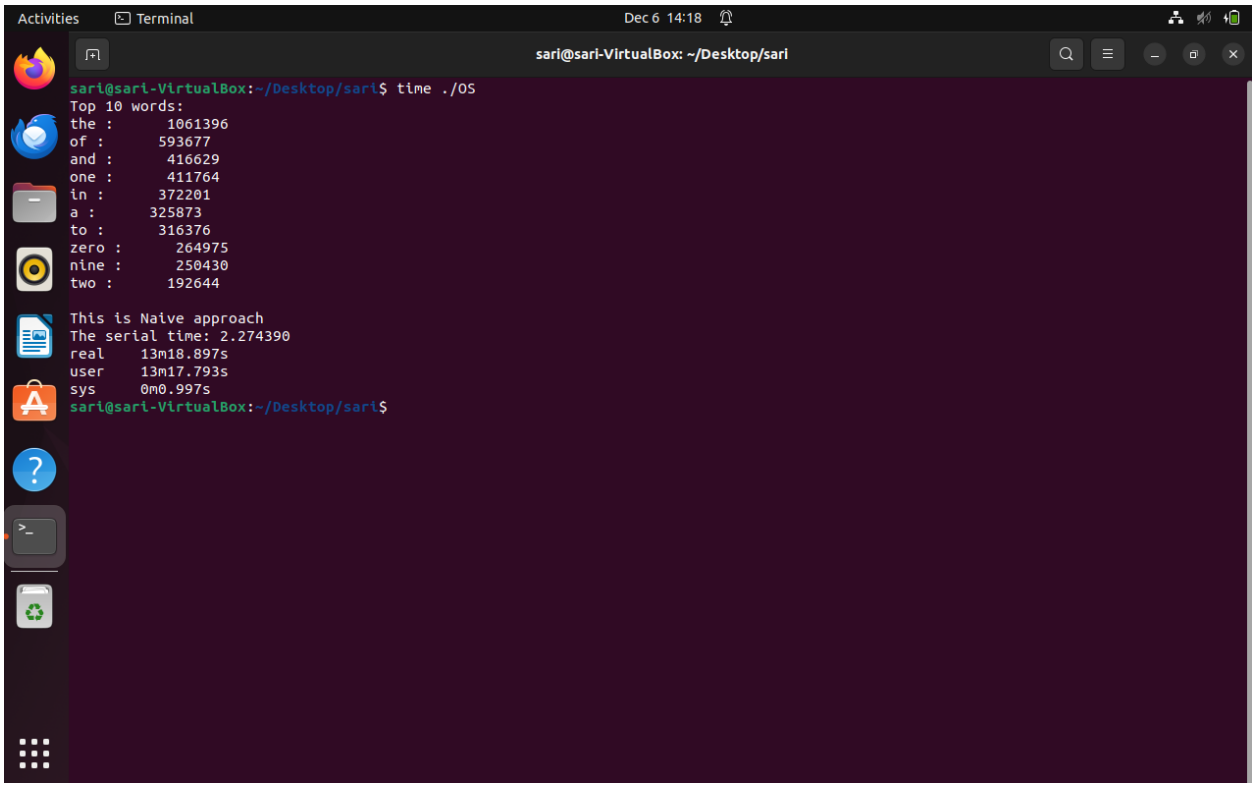


Figure 7: Serial time in naive approach

➔ The serial time is: $2.274390 \times 60 = 136.5s$

➔ All execution time: $(13 \times 60) + 19 = 799s$

➔ The percentage of the serial time: $(136.5/799) \times 100\% = 17\%$

- The maximum speed up can calculate using Amdahl's law:

The formula is $= \frac{1}{S + \frac{(1-s)}{N}}$

Where S is the percentage of the serial part, and N is the number of cores uses.

From the formula maximum speed up $= \frac{1}{17\% + \frac{(1-17\%)}{8}}$ = 3.65

So the maximum speed up gain is 3.65 -1 = 2.65 then the gain is 265%

---

- The optimal number of child processes or threads This refers to determining the best configuration of processes or threads to achieve the maximum efficiency or speedup when running the program, considering the hardware and software environment constraints. It ties into performance analysis and how the available CPU cores and other system resources are utilized effectively

For my laptop the optimal number of threads or child process is 8 because I have 8 cores in my laptop

## 2.4 performance of the 3 approaches:

### 2.4.1 The Naive approach:

For the naïve approach it's should take more of time than multiprocessing or multithreading, because all the code should work in one code in one task so its take a lot of time, the following figure shows the output of the code and the execution time for naïve approach:

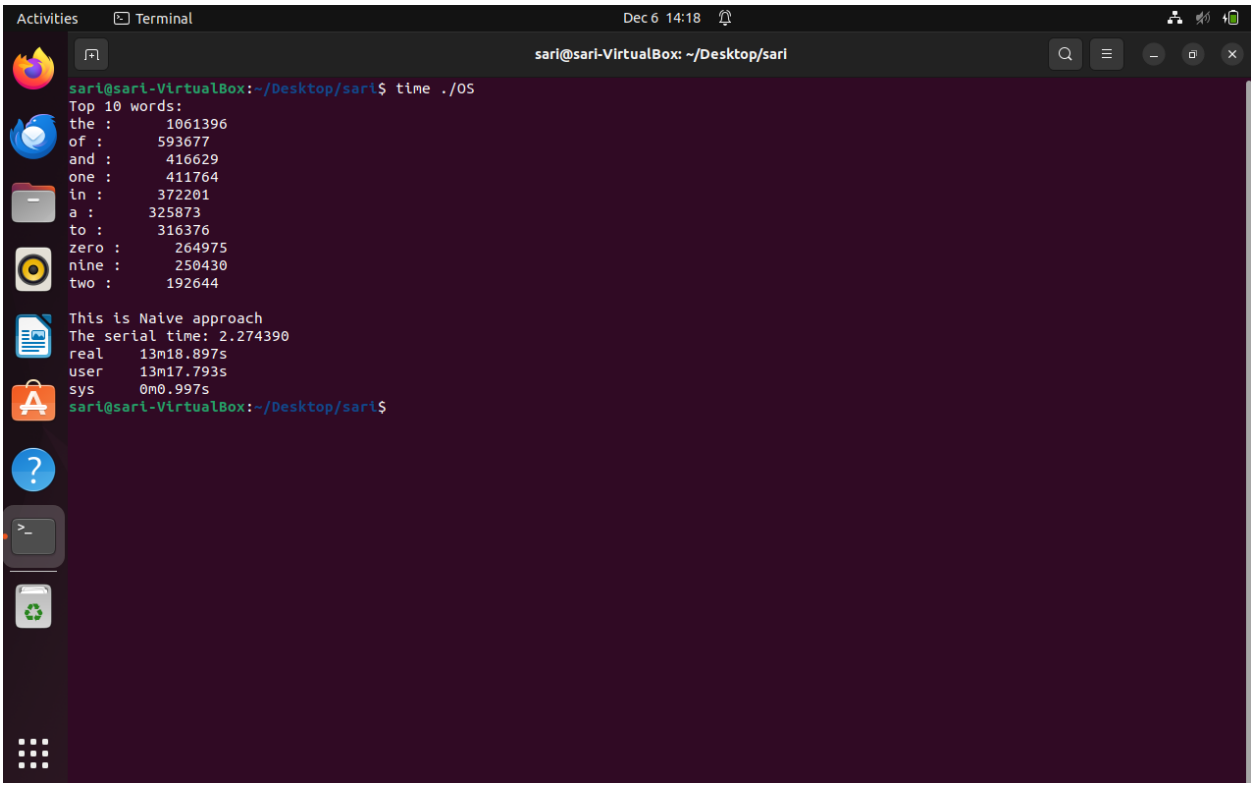


Figure 8: output of naive approach

Note that the naïve approach take 13min + 19 s , so I expect there is no case take a time more than naïve approach.

### 2.4.2 2-child process and 2-thread:

The following figures show the output of 2-child process in addition to its execution time:

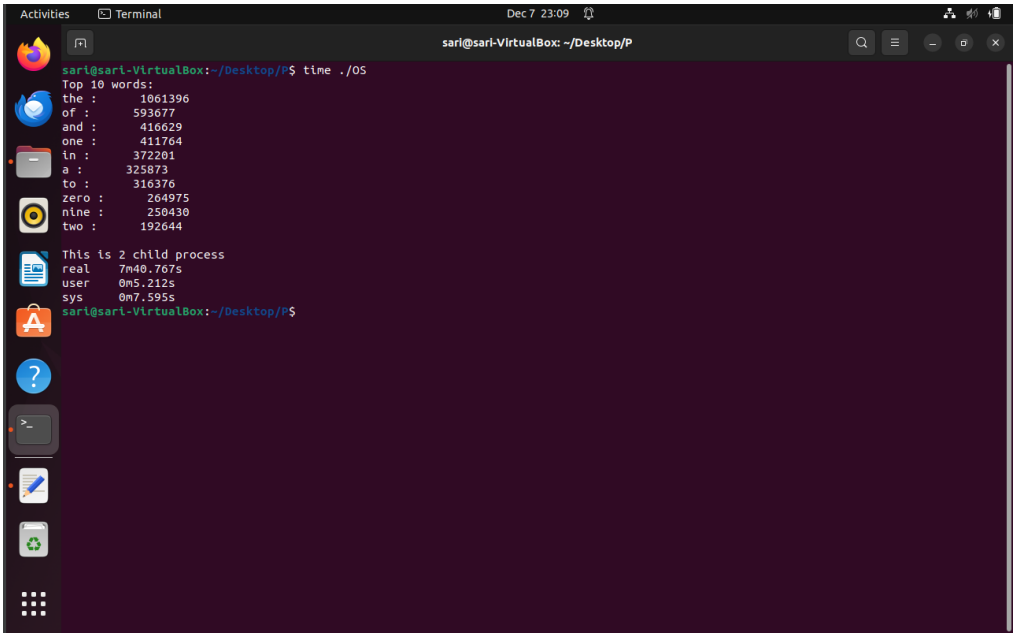


Figure 9: 2-child process

The following figures show the output of 2-thread in addition to its execution time:

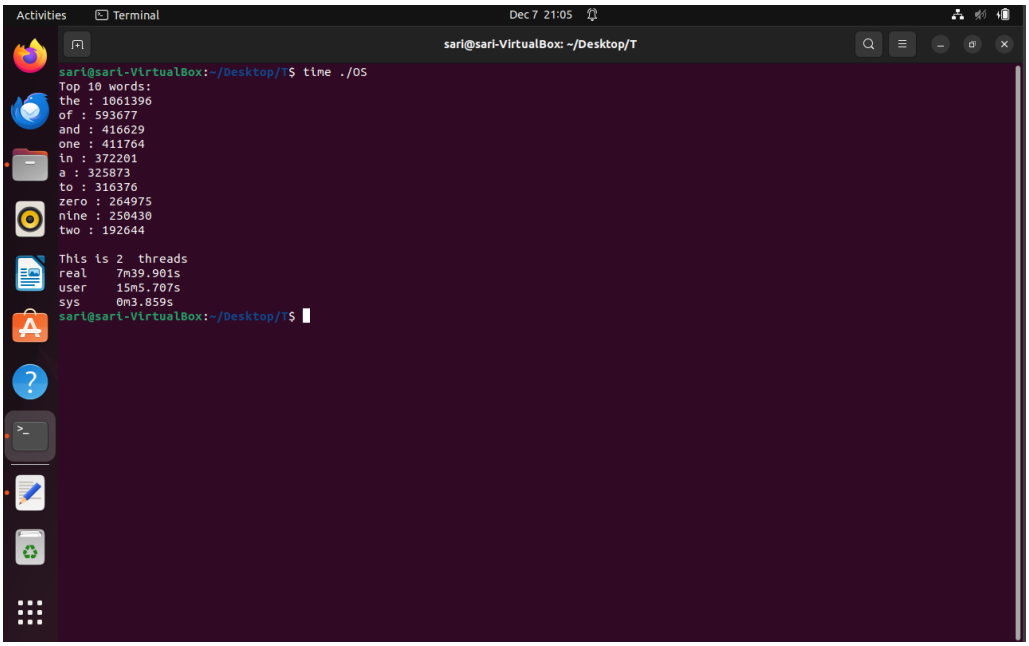


Figure 10: 2-thread

- Note that 2-child process and 2-thread are speed then naive approach because naïve approach run all the code as one task in on core, but in 2-child process and 2-thread its work part of code in 2core.

- In addition, in 2-child process and 2-thread the code divide the code in calculate the frequency of unique words, so in this part the code run each part in core at the same time, this case make my program faster than naïve.

- For 2-child process and 2-thread, the 2-thread should be faster but note that 2 cases are equal, because in 2-thread code I use mutex to lock data which is take more time and in 2-chlid process code I was use pipe which don’t need lock, so the execution time for the 2 cases is so close together .

### 2.4.3 4-child process and 4-thread:

The following figures show the output of 4-child process in addition to its execution time:

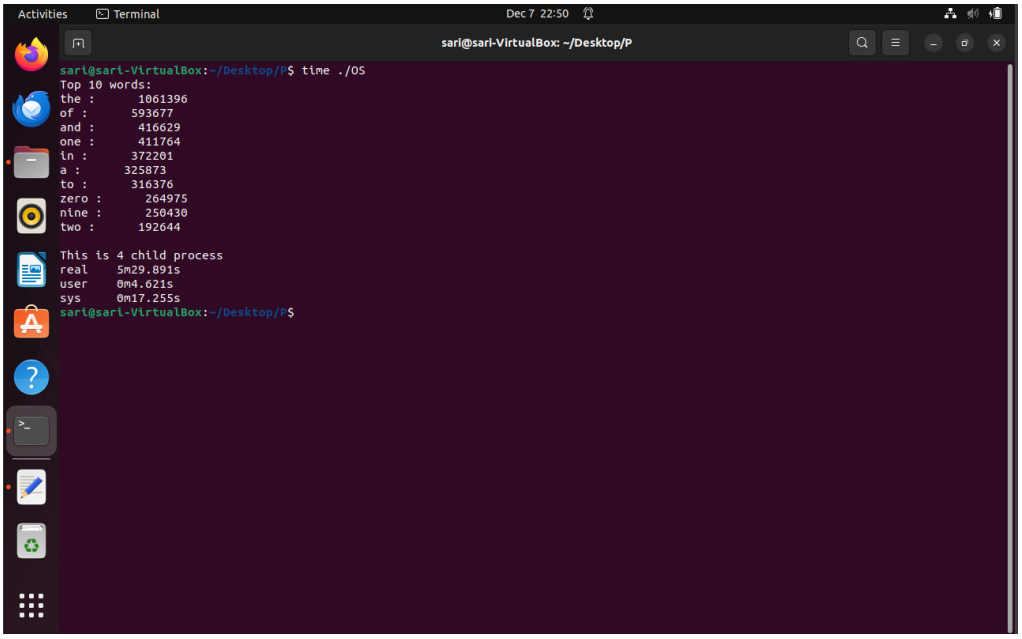


Figure 11: 4-child process

The following figures show the output of 4-thread in addition to its execution time:

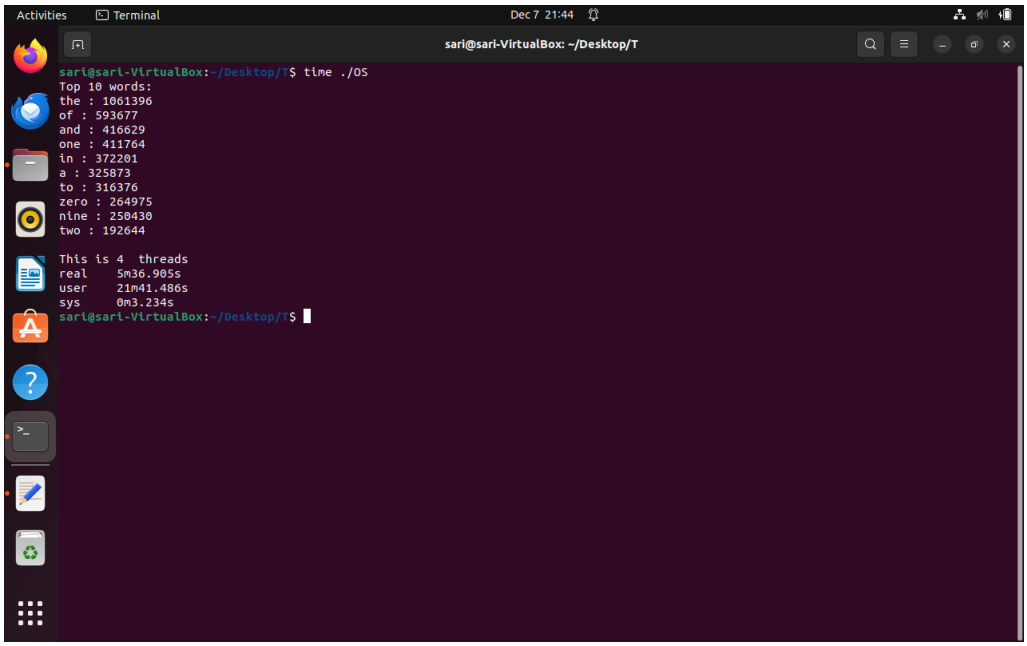


Figure 12: 4-thread

- 4-chlid process take 5min30s and 4-thread 5min37s.
- Note that 4-child process and 4-thread are speed then 2-child process and 2-thread because 2-child process and 2-thread run the parallel part code in 2 cores, but for 4-child process and 4-thread its run the parallel part in 4 cores at the same time

- For 4-child process and 4-thread, the 4-thread should be faster but note that 2 cases are so close, because in 4-thread code I use mutex to lock data which is take more time and in 4-chlid process code I was use pipe which don’t need lock, so the execution time for the 2 cases is so close together .

### 2.4.4 6-child process and 6-thread:

The following figures show the output of 6-child process in addition to its execution time:

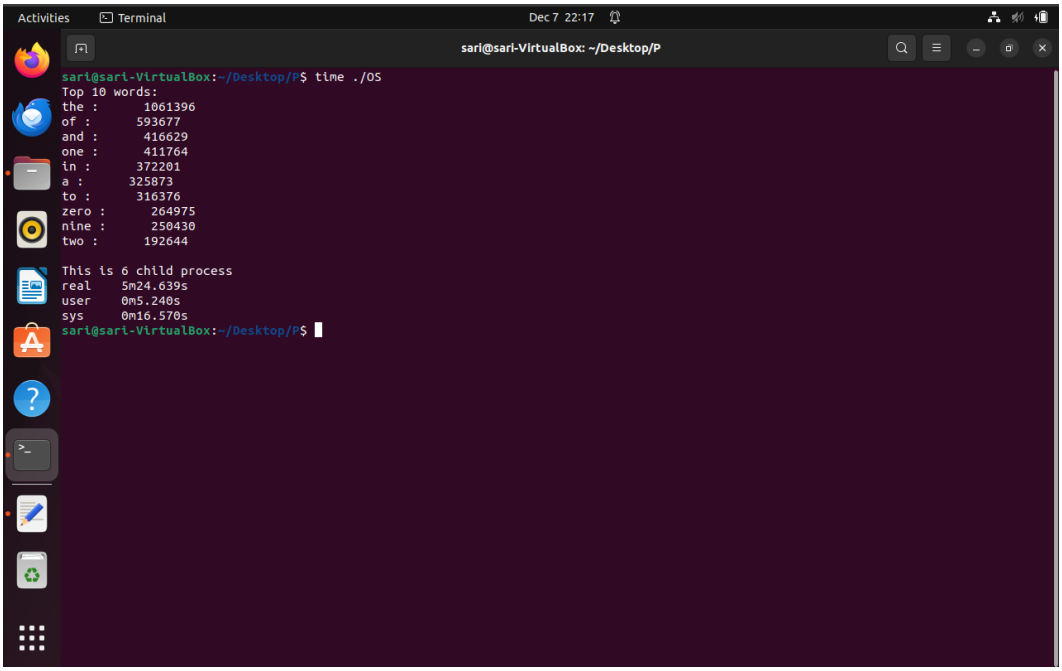


Figure 13: 6-child process

The following figures show the output of 6-thread in addition to its execution time:

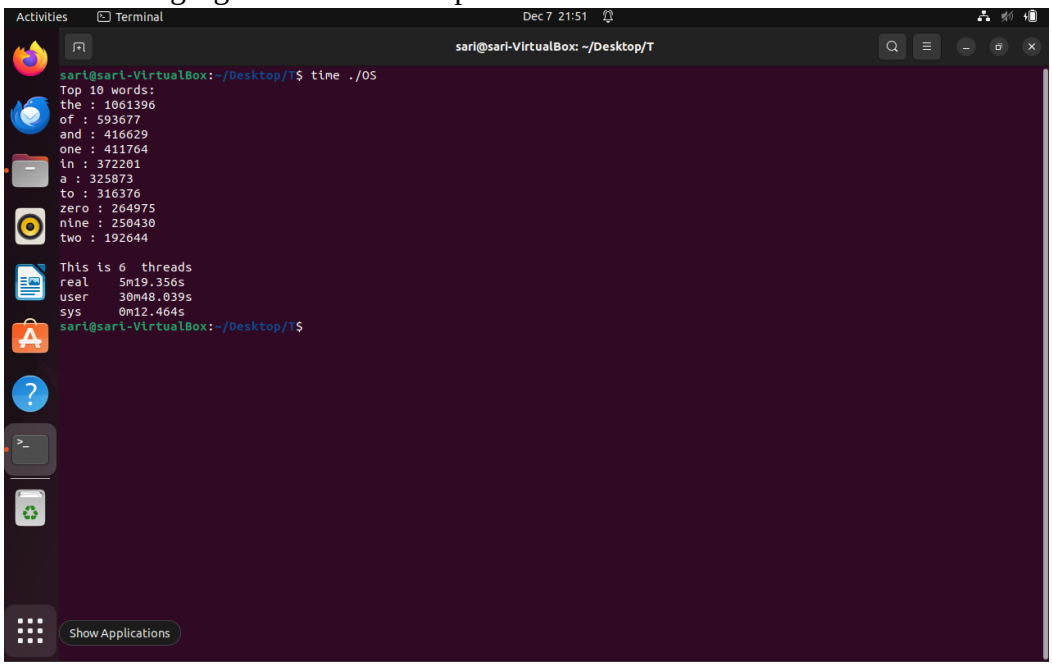


Figure 14: 6-thread

- 6-chlid process take 5min24s and 6-thread 5min19s.
- Note that 6-child process and 6-thread are speed then 4-child process and 4-thread because 4-child process and 4-thread run the parallel part code in 4 cores, but for 6-child process and 6-thread its run the parallel part in 6 cores at the same time.

- Note that 6-thread faster then 6-child process which is I expect, but the difference between them not big because in 6-thread code I use mutex to lock data which is take more time and in 6-chlid process code I was use pipe which don’t need lock, so the execution time for the 2 cases is so close together .

### 2.4.5 8-child process and 8-thread:

The following figures show the output of 8-child process in addition to its execution time:

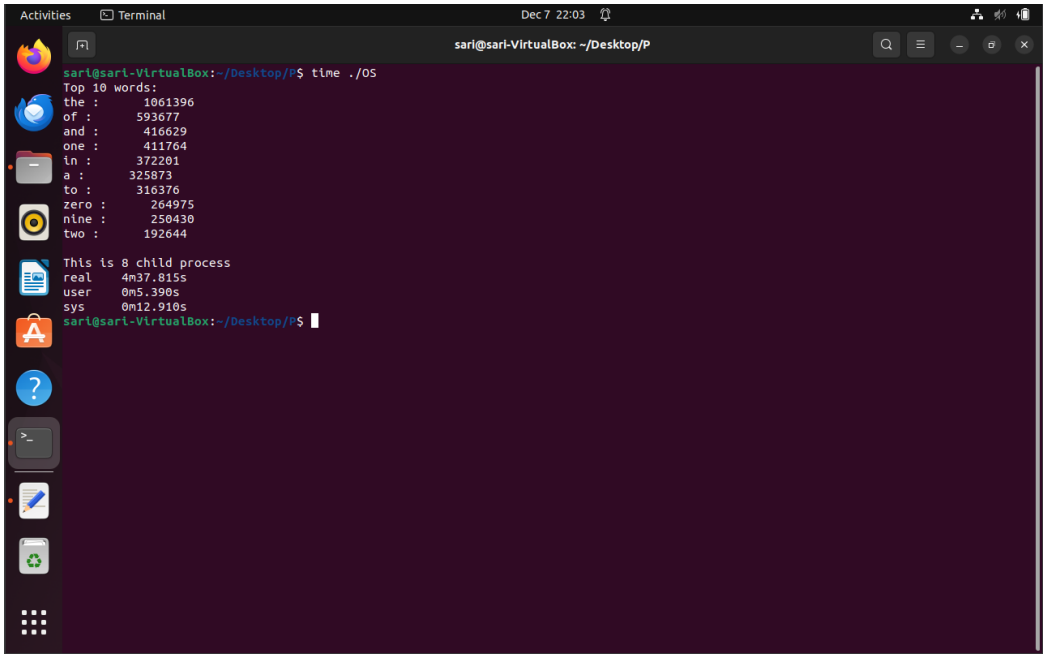


Figure 15: 8-child process

The following figures show the output of 8-thread in addition to its execution time:

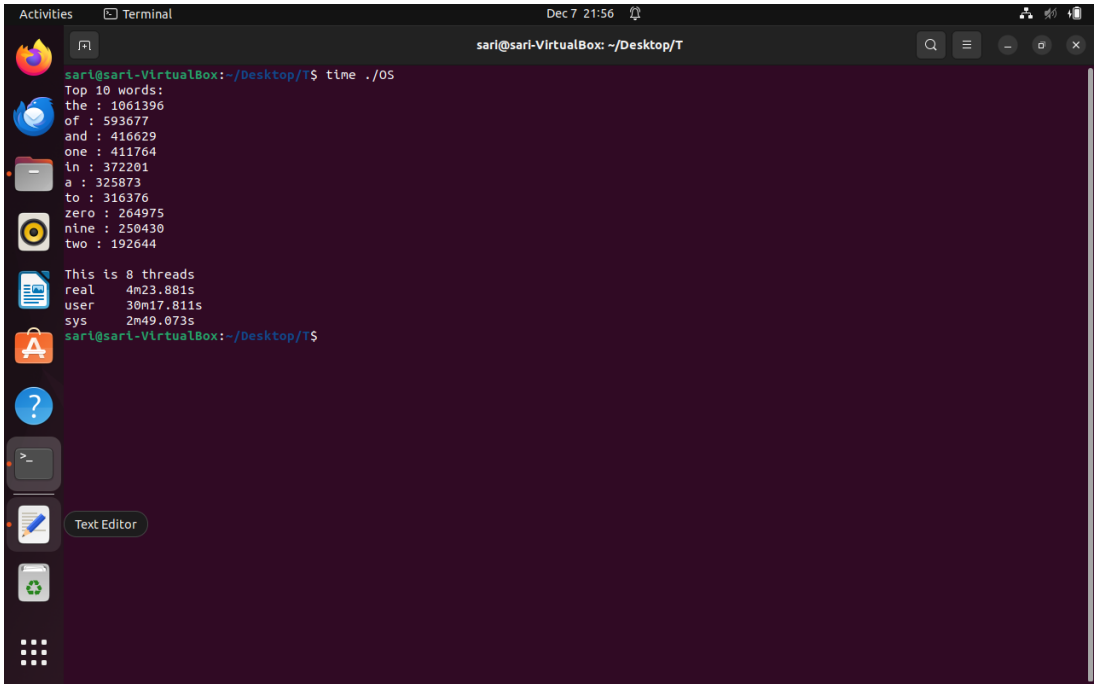


Figure 16: 8-thread

- 8-chlid process take 4min38s and 8-thread 4min24s.
- Note that 8-child process and 8-thread are speed then 6-child process and 6-thread because 6-child process and 6-thread run the parallel part code in 6 cores, but for 8-child process and 8-thread its run the parallel part in 8 cores at the same time.

- Note that in this case and in my my laptop I reached the maximum utilize of processor because I am using all cores in my laptop (8cose), and I reached maximum speed up.

- Note that 8-thread faster then 8-child process which is I expect, but the difference between them not big because in 8-thread code I use mutex to lock data which is take more time and in 8-chlid process code I was use pipe which don’t need lock, so the execution time for the 2 cases is so close together .

### 2.4.6 Summary for all approaches:

The following table show all execution time for all approaches:

| Number of thread or child process | Naive approach | Multiprocessing | Multithreading |
|---|---|---|---|
| **1 (naive)** | 13min18s | -- | -- |
| **2** | -- | 7min40s | 7min40s |
| **4** | -- | 5min30s | 5min37s |
| **6** | -- | 5min24s | 5min19s |
| **8** | -- | 4min28s | 4min24s |

➔ Note that all cases faster than naïve approach, so the parallel code is best than serial code

➔ In addition, when the number of thread or child process increase then the execution time decrease, this note is exactly compiles with Amdahl's law when we increase the number of cores then the speed up will increase according to the law, and we try this low.

# Conclusion

In this project, we explored the power of parallel computing by comparing serial and parallel processing techniques for handling a large data. By implementing multiprocessing and multithreading approaches with increasing numbers of processes and threads, we have shown great importance performance improvements over the naive serial method. Starting from a baseline of 13 minutes and 18 seconds in the serial approach, we progressively reduced execution time to around 4 minutes and 24 second by utilizing all 8 cores of the laptop. The results validated Amdahl's law, showing that as we increased the number of cores, the processing speed improved substantially, with performance gains reaching up to 265%.

# References

[1] https://www.indeed.com/career-advice/career-development/multithreading-vs-multiprocessing